

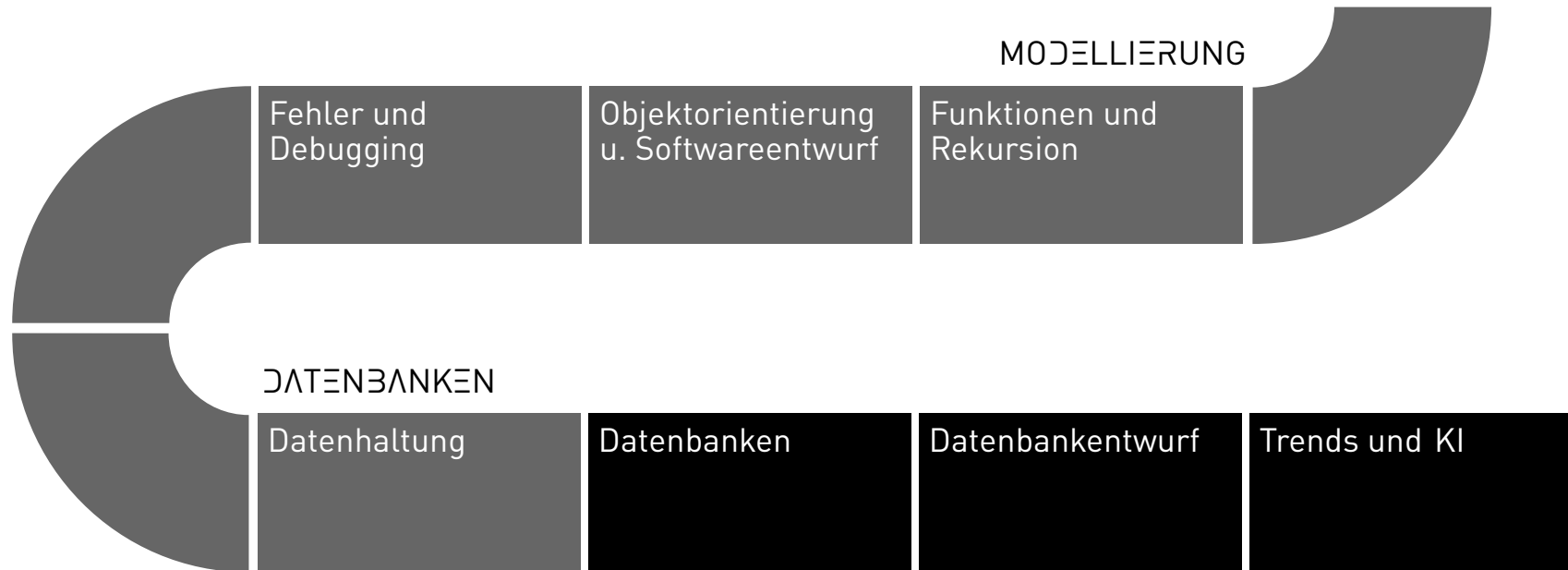
# Programming and Databases

Joern Ploennigs

Modularization

MIDJOURNEY: MODULAR BLOCKS, REF. PIET MONDRIAN

# PROCESS



## ORGANIZING PROJECTS ACROSS MULTIPLE FILES

Challenges of large programs:

- Dozens of classes with hundreds of functions
- Thousands of lines of code
- Difficult to keep an overview
- Version conflicts when multiple programmers work on it

Solution - Split code into .py files:

- *Per class* one file (for class definitions)
- *Per topic* one file (e.g., math functions)
- *Per domain* one file (e.g., loading vs. processing)

## THE MAIN() FUNCTION AS THE STANDARD ENTRY POINT

Where does Python know what to run?

- Declare a specific file as the project's *center*
- This contains a special function named `main()`
- Exists in almost every programming language
- Specifies the *start point of the program*
- Can receive no arguments or dynamic arguments (command-line arguments)

# THE MAIN() FUNCTION IN PYTHON

Different ways to run:

1. As a script on the command line / standalone program
2. Imported into an interactive Python console
3. Imported into another Python file

Problem: In case 1 we want to use the entire script, in case 2+3 only parts

The solution: the **name** variable:

```
def main():  
    print("This is the main  
function")  
  
if __name__ == "__main__":  
    main()    # nur bei direktem Aufruf
```

## BEST PRACTICE FOR THE MAIN() FUNCTION

What belongs BEFORE the main() definition:

✓ Only allowed:

- Function definitions
- Class definitions

Advantages of this structure:

- The program flow becomes clearer
- The program is easy to modify
- The program is more reusable

✗ Avoid:

- Variable assignments (global variables)
- Function calls (side effects)

## PROGRAM FLOW ACROSS MULTIPLE FILES

- We now have a central starting point.
- How do we call code from other files?
- Every programming language has commands to load external code:
  - `import` and `include` (Python)
  - Special 'header files' that define the relationships between files

# PROGRAM FLOW IN PYTHON - IMPORT STATEMENT

Strategy:

- A .py file contains `main()` (Case 1)
- All others without `main()` or with `__name__` are ignored (Case 3)
- The `import` statement loads external files
- *Warning:* The entire code is executed, including variables and function calls!

```
import external_file

def main():
    print("This is the main
function")
```

Variables/functions/classes from `external_file.py` are stored in the object `external_file`



# IMPORTING EXTERNAL LIBRARIES (PACKAGES)

*Terminology:*

- *Module*: Imported scripts
- *Namespace*: The resulting object (type: module)
- *Package*: A module with additional submodules

# FUNDAMENTALS OF PROJECT STRUCTURE

## General Structure

```
main.py [contains main()]  
├─ module1.py  
├─ module2.py  
└─ helpers.py
```

Code in `main.py`

```
import module1  
import module2  
import helpers
```

## Example - City Project

```
city.py  
├─ buildings.py  
├─ streets.py  
└─ geometry.py
```

Code in `main.py`

```
import buildings  
import streets  
import geometry
```

## THE IMPORT STATEMENT: ADVANCED USE CASES

- Python's import system is complex and multifaceted.
- In day-to-day use, however, you typically encounter only the following additional constructs:
  - `from-import` statement: Imports a submodule or a portion of a module directly, without the parent namespace
  - `from-import-as` statement: Works the same, but renames the imported object

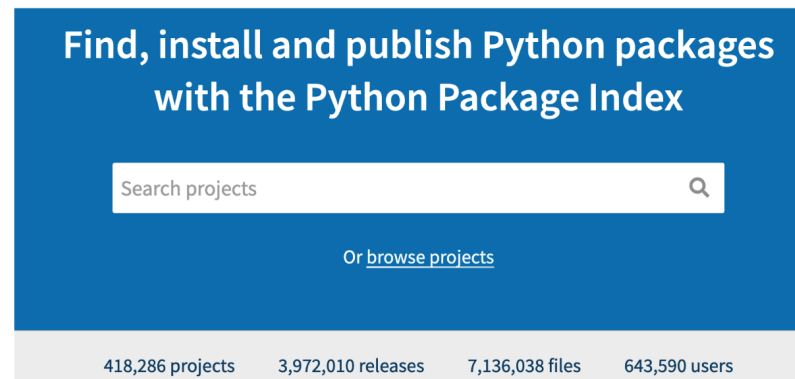
```
from external_file import external_function as ext_func
```

## IMPORT STATEMENTS – THE GATEWAY TO THE WORLD

- We can import not only our own packages, but also packages published by other people/organizations!
- There are several ways to access packages:
- The package is included with Python
- Download directly as .py files / folders containing .py files
- Use a package manager (pip, uv) (Better!)

# PACKAGES - THE REAL POWER OF PYTHON

- Python's position as the world's most popular programming language is largely based on its evolution into an interface and pipeline language.
- Almost every computer-solvable problem can be solved by cleverly chaining together the right Python packages.
- The official hub for packages, the Python Package Index (PyPI), currently hosts over 418,000 freely available packages.
- These can in most cases be installed with a single command or click



## LESSON LEARNED

How do you prioritize correctly?



“I have two kinds of problems, the urgent and the important. The urgent are not important, and the important are never urgent.”, Eisenhower

## LESSON LEARNED - PRIORITIZATION

- With the Eisenhower matrix, tasks are divided into urgent and important
- You first tackle the urgent and important tasks



Questions?



